

DATA-260 — Homework 4

Pinal Pawar (pinal.pawar@sjsu.edu)

Section 0 — Configuration

SID4	7875
PORT_BASE	8675
PREFIX	s7875
SEED	7875
VERIFY_SEED	267875
DOMAIN_ID	3 (grocery supply and recall notices)
Hardware	MacBook Air, Apple Silicon (arm64), macOS
Database	MySQL 26.7.0 (Homebrew), database s7875_rel
Embedding model	sentence-transformers/all-MiniLM-L6-v2 (HuggingFace, via LlamaIndex)
Generation model (Part 4)	Ollama qwen3:1.7b, local inference
GitHub repository	https://github.com/PinalPawar/data260-7875 (collaborators: Sbnikitha, supriyaselvangesan)
Git commit (tagged hw4)	ba4e94f548e87cff81468de01c9c150808975b2d

Part 1 — React Client

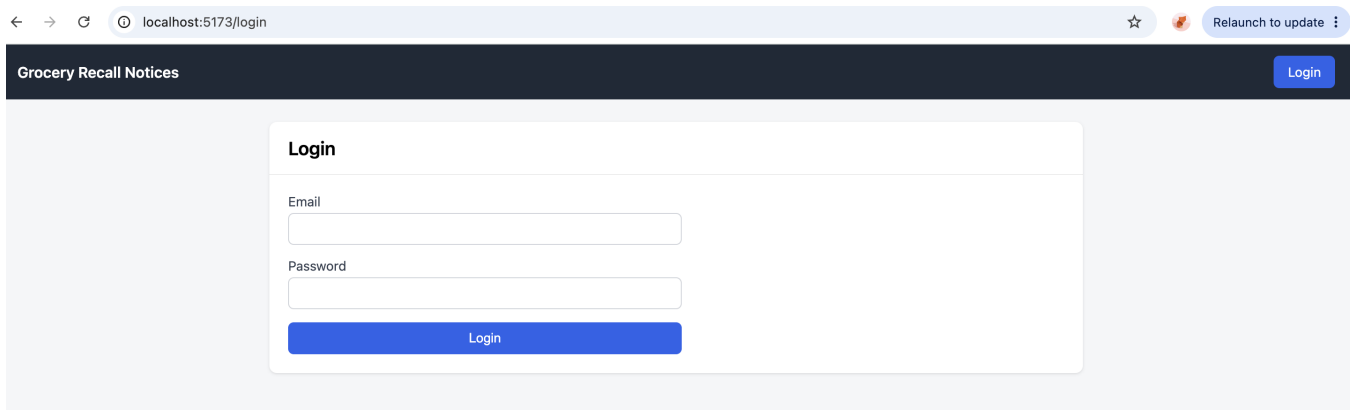
Full CRUD cycle verified live in the browser at <http://localhost:5173>, backed by the FastAPI/MySQL backend on port 8675. Login gates all record access; Create, Update and Delete all persist through the backend API and redirect back to Home showing the updated list. One code snippet + output screenshot per component, as required.

frontend/src/pages/Login.jsx — email+password login; the backend sets the HttpOnly session cookie itself, this component only records who's logged in.

```
// frontend/src/pages/Login.jsx -- email+password login (HW4 Part 1 login requirement)
export default function Login({ onLoggedIn }) {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [error, setError] = useState("");
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    setError("");
    try {
      const res = await login(email, password); // POST /api/auth/login; backend sets the HttpOnly cookie
      onLoggedIn(res.user);
      navigate("/");
    } catch (err) {
      setError(err.response?.status === 401 ? "Invalid email or password."
        : "Login failed. Is the backend running on port 8675?");
    }
  }

  return (
    <form className="form" onSubmit={handleSubmit}>
      <label>Email
        <input type="email" value={email} onChange={(e) => setEmail(e.target.value)} required />
      </label>
      <label>Password
        <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} required />
      </label>
      {error && <div className="notice error">{error}</div>}
      <button className="btn primary" type="submit">Login</button>
    </form>
  );
}
```



The screenshot shows a web browser window with the address bar displaying 'localhost:5173/login'. The page features a dark header bar with the text 'Grocery Recall Notices' on the left and a blue 'Login' button on the right. Below the header, the main content area contains a white 'Login' form. The form has two input fields: 'Email' and 'Password', both of which are empty. Below these fields is a blue 'Login' button. The overall layout is clean and modern.

Output: Login page (empty form).

frontend/src/pages/CreateRecord.jsx — primary/secondary input fields + submit button, per spec Section II.

```
// frontend/src/pages/CreateRecord.jsx -- rendered on /create (HW4 Part 1, Section II)
export default function CreateRecord({ onAdd }) {
  const [form, setForm] = useState({ product: "", manufacturer: "", email: "",
    description: "", category: CATEGORIES[0] });
  function set(field) { return (e) => setForm({ ...form, [field]: e.target.value }); }

  async function handleSubmit(e) {
    e.preventDefault();
    await onAdd(form); // parent (App.jsx) POSTs to the backend, then redirects to "/"
  }
  return (
    <form className="form" onSubmit={handleSubmit}>
      <label>Product (primary field)
        <input value={form.product} onChange={set("product")} required />
      </label>
      <label>Manufacturer (secondary field)
        <input value={form.manufacturer} onChange={set("manufacturer")} required />
      </label>
      <button className="btn primary" type="submit">Submit Recall Notice</button>
    </form>
  );
}
```

localhost:5173/create

Relaunch to update

Grocery Recall Notices

+ Add Record Pinal Pawar Logout

Report a Recall

Product (primary field)

Frozen Spinach

Manufacturer (secondary field)

GreenFields Co

Contact email

qa@greenfields.com

Description

Possible Listeria contamination in lot GF-2291.

Category

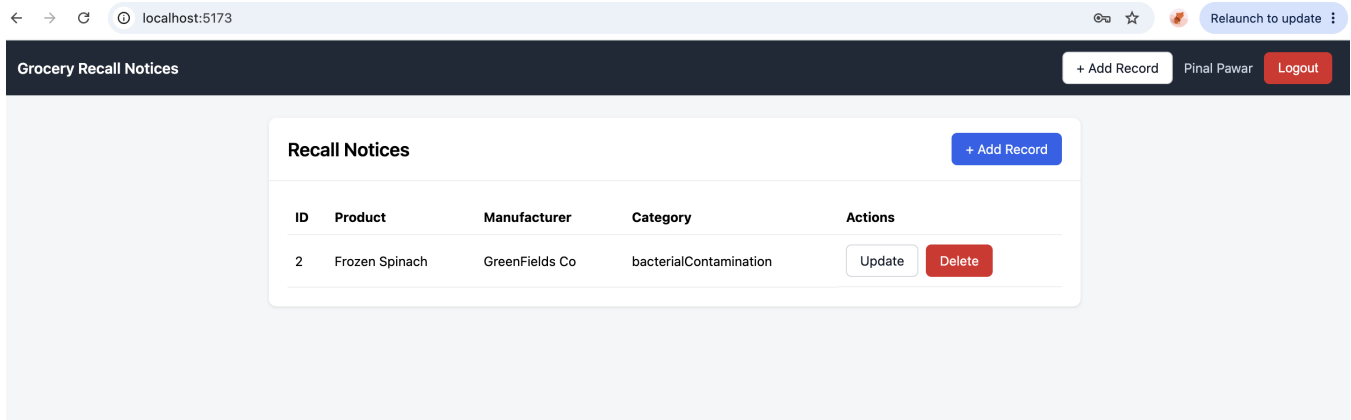
bacterialContamination

Submit Recall Notice

Output: Create form filled in with a new recall notice.

frontend/src/pages/Home.jsx — login-gated render path + record table, per spec Section I.

```
// frontend/src/pages/Home.jsx -- rendered on / (HW4 Part 1, Section I): displays all records
export default function Home({ auth, notices, loading, onDelete }) {
  if (!auth.loggedIn) {
    return <div className="notice">Login required to view recall notices.</div>;
  }
  return (
    <table className="table">
      <tbody>
        {notices.map((n) => (
          <tr key={n.id}>
            <td>{n.id}</td><td>{n.product}</td><td>{n.manufacturer}</td>
            <td className="actions">
              <Link to={`/update/${n.id}`}>Update</Link>
              <Link to={`/delete/${n.id}`}>Delete</Link>
            </td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```



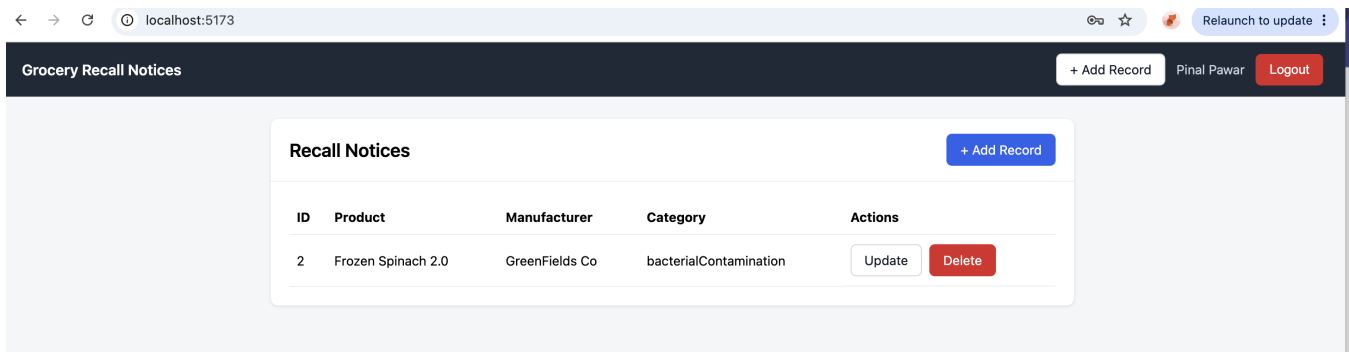
Output: Home page showing the newly created record (id 2).

frontend/src/pages/UpdateRecord.jsx — loads the existing record by id from the URL, then PUTs the edited fields, per spec Section III.

```
// frontend/src/pages/UpdateRecord.jsx -- rendered on /update/:id (HW4 Part 1, Section III)
export default function UpdateRecord({ onUpdate }) {
  const { id } = useParams();
  const noticeId = Number(id);
  const [form, setForm] = useState(null);

  useEffect(() => {
    (async () => setForm(await fetchNoticeById(noticeId)))();
  }, [noticeId]);

  function set(field) { return (e) => setForm({ ...form, [field]: e.target.value }); }
  async function handleSubmit(e) {
    e.preventDefault();
    await onUpdate(noticeId, form); // PUT to the backend, then redirect to "/" showing the updated list
  }
  if (!form) return <div className="notice">Loading...</div>;
  return (
    <form className="form" onSubmit={handleSubmit}>
      <label>Product (primary field)
        <input value={form.product} onChange={set("product")} required />
      </label>
      <label>Manufacturer (secondary field)
        <input value={form.manufacturer} onChange={set("manufacturer")} required />
      </label>
      <button className="btn primary" type="submit">Save Changes</button>
    </form>
  );
}
```



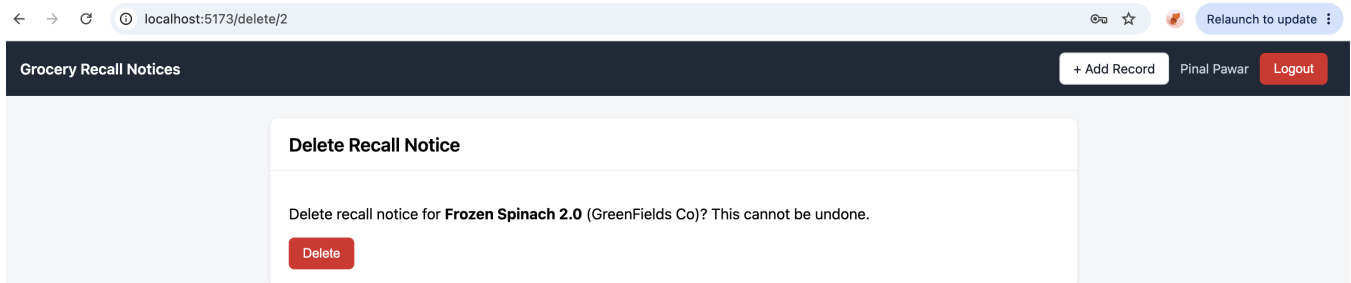
Output: Home page after Update — product changed to "Frozen Spinach 2.0".

frontend/src/pages/DeleteRecord.jsx — shows what's about to be deleted before confirming, per spec Section IV.

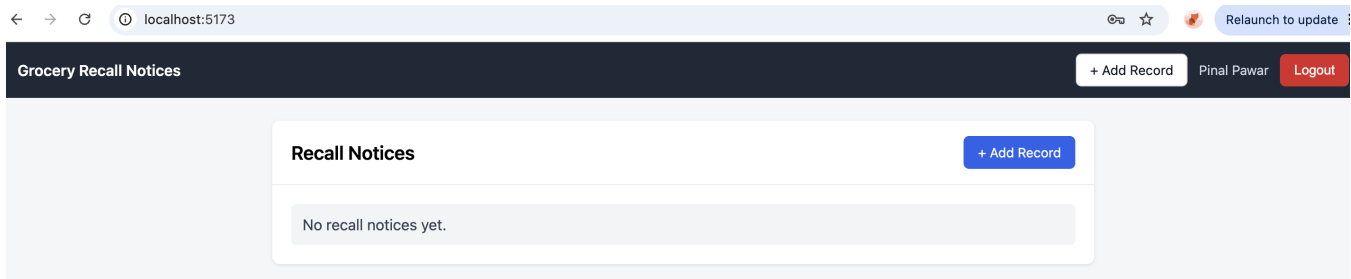
```
// frontend/src/pages/DeleteRecord.jsx -- rendered on /delete/:id (HW4 Part 1, Section IV)
export default function DeleteRecord({ onDelete }) {
  const { id } = useParams();
  const noticeId = Number(id);
  const [notice, setNotice] = useState(null);

  useEffect(() => {
    (async () => setNotice(await fetchNoticeById(noticeId)))();
  }, [noticeId]);

  async function handleDelete() {
    await onDelete(noticeId); // DELETE to the backend, then redirect to "/"
  }
  return notice && (
    <>
      <p>Delete recall notice for <strong>{notice.product}</strong> ({notice.manufacturer})?
        This cannot be undone.</p>
      <button className="btn danger" onClick={handleDelete}>Delete</button>
    </>
  );
}
```



Output: Delete confirmation page.



Output: Home page after Delete — list empty again.

Part 2 — MySQL Persistence and Server-Side Sessions

Database `s7875_rel` created with tables `notices`, `users` and `sessions`. The session dependency is named `db_session_basede26` throughout (strict naming requirement). Login issues an opaque random token stored server-side in the `sessions` table; the browser only ever holds that token in an `HttpOnly` cookie (`s7875_session`) — no user data lives in the cookie itself. All five CRUD operations (`POST/GET-all/GET-by-id/PUT/DELETE`) were exercised in Postman; a 401 “Login required” was confirmed when logged out.

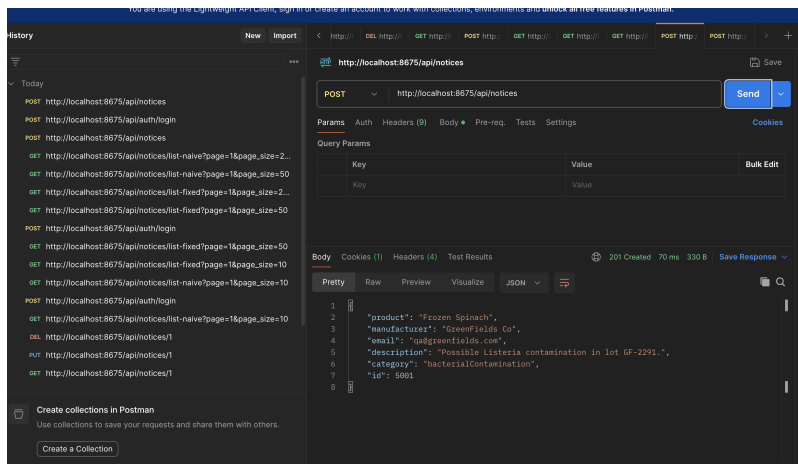
routers/session_auth.py — the cookie holds only the opaque token; who it belongs to and when it expires lives server-side in the sessions table, looked up on every request.

```
# routers/session_auth.py -- login issues an opaque token, stored server-side
@router.post("/login")
def login(payload: LoginPayload, response: Response,
          db_session_basede26: Session = Depends(get_db_session_basede26)):
    user = db_session_basede26.query(User).filter(User.email == payload.email).first()
    if not user or not verify_password(payload.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Invalid email or password")

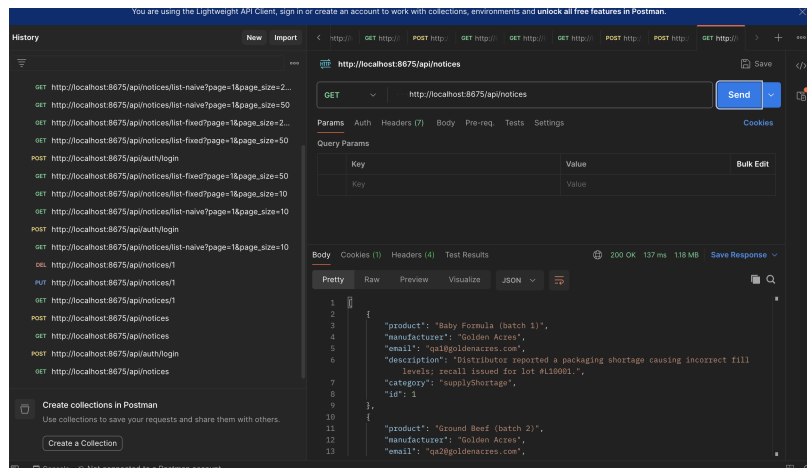
    token = secrets.token_hex(32) # opaque: unguessable, carries no meaning on its own
    expires = datetime.utcnow() + timedelta(minutes=SESSION_TTL_MINUTES)
    db_session_basede26.add(SessionToken(id=token, user_id=user.id, expires_at=expires))
    db_session_basede26.commit()

    response.set_cookie(key=SESSION_COOKIE_NAME, value=token,
                        httponly=True, samesite="lax", max_age=SESSION_TTL_MINUTES * 60)
    return {"message": "logged in", "user": {"id": user.id, "name": user.name, "email": user.email}}

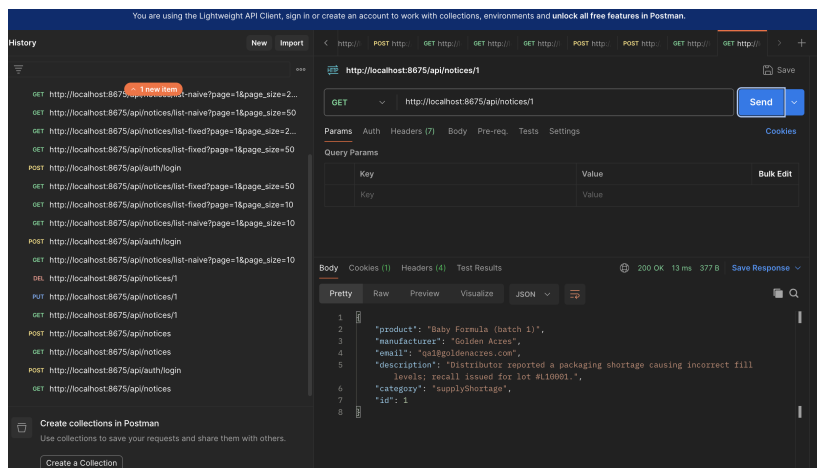
def require_session(request: Request,
                    db_session_basede26: Session = Depends(get_db_session_basede26)) -> User:
    """Protects the notice CRUD endpoints: looks up the cookie's token in the
    sessions table server-side and returns the User, or raises 401."""
    token = request.cookies.get(SESSION_COOKIE_NAME)
    if not token:
        raise HTTPException(status_code=401, detail="Login required")
    session_row = db_session_basede26.query(SessionToken).filter(SessionToken.id == token).first()
    if not session_row:
        raise HTTPException(status_code=401, detail="Login required")
    if session_row.expires_at < datetime.utcnow():
        db_session_basede26.delete(session_row)
        db_session_basede26.commit()
        raise HTTPException(status_code=401, detail="Session expired, please log in again")
    return db_session_basede26.query(User).filter(User.id == session_row.user_id).first()
```



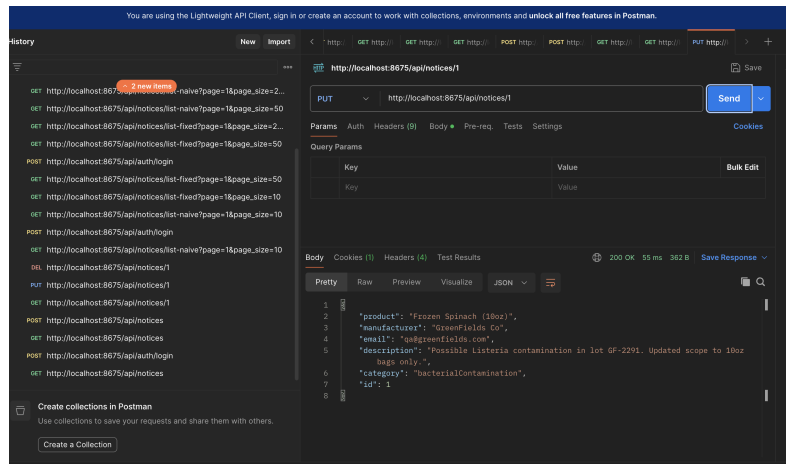
Postman: POST /api/notices -> 201 Created, returns the new record (id 5001).



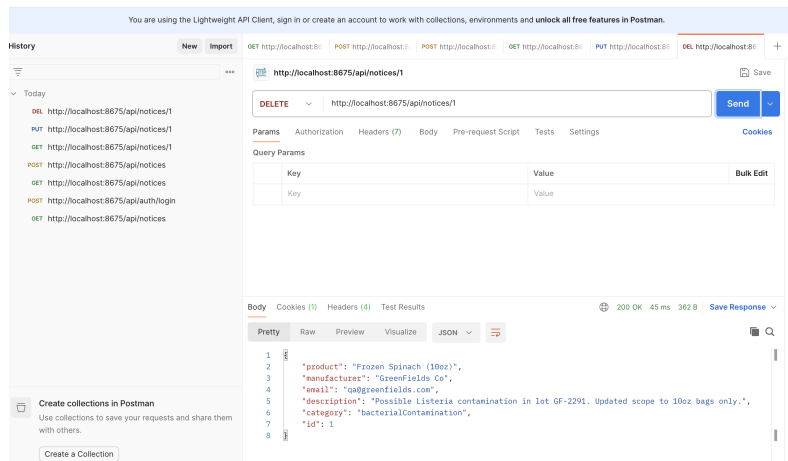
Postman: GET /api/notices -> 200 OK, returns the array of all notice records.



Postman: GET /api/notices/1 -> 200 OK, returns the single record with id 1.



Postman: PUT /api/notices/1 -> 200 OK, returns the updated record.



Postman: DELETE /api/notices/1 -> 200 OK, returns the deleted record.

```

~/Downloads/data260-7875-main — python3 main.py
Last login: Sun Sep 27 13:30:17 on ttys009
(base) atharvamdada@Atharvas-MacBook-Air data260-7875-main % cd ~/Downloads/data260-7875-main
(base) atharvamdada@Atharvas-MacBook-Air data260-7875-main % mysql -u root -p s7875_rel -e "SELECT * FROM notices; SELECT * FROM users; SELECT * FROM sessions;"
Enter password:
+-----+-----+-----+-----+
| id | name | email | password_hash |
+-----+-----+-----+-----+
| 1 | Pinal Pawar | pinal@example.com | $2b$12$SHA8jL4qL.9hSz1NPaWqPkuwLtA/N/3YFrkE9kuZNBbPUH14xM..6G |
+-----+-----+-----+-----+
+-----+-----+-----+-----+
| id | user_id | created_at | expires_at |
+-----+-----+-----+-----+
| 306c0edfe089c92b35f4b091feb47f6db2c5ec657fa5eb4f32d1667a6411e778 | 1 | 2026-09-27 13:59:17 | 2026-09-27 21:29:17 |
+-----+-----+-----+-----+
(base) atharvamdada@Atharvas-MacBook-Air data260-7875-main %

```

Database screenshot: users table shows a bcrypt password hash (\$2b\$12\$...); sessions table shows a token row tied to user_id with a 30-minute created_at/expires_at gap.

Name	Size	Kind
data260-7875-main	--	Folder
rag.py	18 KB	Python File
__pycache__	--	Folder
frontend	--	Folder
schemas.py	555 bytes	Python File
query_counter.py	993 bytes	Python File
models.py	3 KB	Python File
database.py	1 KB	Python File
crud.py	1 KB	Python File
templates	--	Folder
src	--	Folder
scripts	--	Folder
screenshots	--	Folder
schema_agent.py	4 KB	Python File
run_nondeterminism.py	6 KB	Python File
run_experiment3.py	2 KB	Python File
run_experiment2.py	2 KB	Python File
run_experiment1.py	2 KB	Python File
routers	--	Folder
retrieval_pipeline.py	11 KB	Python File
requirements.txt	511 bytes	Plain Text
reports	--	Folder
questions.yaml	3 KB	YAML
nginx.conf	303 bytes	Document
natural_retry_test.py	629 bytes	Python File
main.py	6 KB	Python File
index.html	11 KB	HTML text
hw1_client.py	2 KB	Python File
graph_agent.py	6 KB	Python File
corpus	--	Folder
agents_demo.py	11 KB	Python File
SOURCES.md	7 KB	md
README.md	7 KB	md
Dockerfile	387 bytes	Document
DOMAIN_SCHEMA.md	361 bytes	md
CORPUS_MANIFEST.json	13 KB	JSON
AGENT.md	588 bytes	md

Project folder structure (root-level, shared codebase extended from HW1-3).

Part 3 — N+1 Query Problem and Query Tuning

5,000 notices and 200 related_info rows were seeded (SEED=7875), generated by scripts/seed_hw04_part3.py; schema defined in models.py (RelatedInfo), created via Base.metadata.create_all(). Each of the 200 related rows is tied to one of 200 distinct notice IDs, and both list endpoints include this related data in their response. list-naive issues one query per notice (N+1); list-fixed uses a single JOIN via SQLAlchemy joinedload(). 30 requests per configuration at page sizes 10/50/200 (180 requests total) were timed and their SQL statement counts logged via a thread-local counter; raw per-request data for all 180 requests is in reports/hw04/raw/n1_raw.{json,csv}.

scripts/seed_hw04_part3.py — real output below, from RUN_LOG.txt:

```
# scripts/seed_hw04_part3.py -- step 1: reproducible seed (SEED=7875=SID4)
if __name__ == "__main__":
    rng = random.Random(SEED)
    RelatedInfo.__table__.drop(bind=engine, checkfirst=True) # wiped first so the
    Notice.__table__.drop(bind=engine, checkfirst=True)      # seed is reproducible run-to-run
    Base.metadata.create_all(bind=engine)

    notice_rows = build_notices(rng, 5000)
    db_session_based26.bulk_insert_mappings(Notice, notice_rows)
    db_session_based26.commit()

    notice_ids = [row[0] for row in db_session_based26.query(Notice.id).all()]
    related_rows = build_related_rows(rng, notice_ids, 200) # 200 related rows, each tied
    db_session_based26.bulk_insert_mappings(RelatedInfo, related_rows) # to a distinct notice id
    db_session_based26.commit()
    print(f"Seeded {len(notice_rows)} notices and {len(related_rows)} related_info rows (SEED={SEED}).")
```

Terminal output (reports/hw04/RUN_LOG.txt).

```
$ python3 scripts/seed_hw04_part3.py
Seeded 5000 notices and 200 related_info rows (SEED=7875).
```

Page size	Version	SQL stmts/req	p50 (ms)	p95 (ms)	p99 (ms)
10	naive	11	14.89	21.06	36.14
10	fixed	1	6.61	8.22	9.03
50	naive	51	38.91	49.06	54.07
50	fixed	1	7.48	9.36	31.50
200	naive	201	130.41	147.77	152.59
200	fixed	1	10.46	11.37	36.06

Speed-up (naive p50 / fixed p50): **2.25x** at page size 10 (14.89ms → 6.61ms), **5.20x** at page size 50 (38.91ms → 7.48ms), **12.46x** at page size 200 (130.41ms → 10.46ms).

Why the speed-up grows with page size: the fixed version always runs exactly 1 SQL statement no matter how many rows are requested, so its p50 latency stays low and grows only modestly as page size grows (6.61ms at page size 10 to 10.46ms at page size 200 — about 1.6x over a 20x increase in rows, since the single query itself still has to return more rows). The naive version runs one additional query per row, so each extra row adds roughly one more query-execution/round-trip cost on top of the base cost, and its p50 latency scales close to linearly with page size (14.89ms to 130.41ms, about 8.8x over the same 20x increase in rows). Because naive latency grows far faster than fixed latency as page size increases, the ratio between them — the speed-up — necessarily grows larger as page size grows: the per-query overhead that N+1 pays scales with N, while the fixed version pays that overhead only once, regardless of N.

scripts/add_index_part3.py — real output below, from reports/hw04/raw/explain_before_after.txt:

```
# scripts/add_index_part3.py -- step 8: EXPLAIN before/after adding the index
SAMPLE_QUERY = "SELECT * FROM related_info WHERE notice_id = 4242"
with engine.connect() as conn:
    print(explain(conn, "BEFORE adding index"))
    conn.execute(text("CREATE INDEX idx_related_info_notice_id ON related_info(notice_id)"))
    conn.commit()
    print(explain(conn, "AFTER adding index idx_related_info_notice_id"))
```

Real MySQL EXPLAIN output (tree format).

```
--- EXPLAIN BEFORE adding index ---
Query: SELECT * FROM related_info WHERE notice_id = 4242
-> Filter: (related_info.notice_id = 4242) (cost=20.2 rows=20)
    -> Table scan on related_info (cost=20.2 rows=200)

--- EXPLAIN AFTER adding index idx_related_info_notice_id ---
Query: SELECT * FROM related_info WHERE notice_id = 4242
-> Index lookup on related_info using idx_related_info_notice_id (notice_id = 4242) (cost=0.35 rows=1)
```

BEFORE: a full table scan — MySQL has no supporting index on `notice_id` (it's intentionally a plain column, not a ForeignKey, so InnoDB doesn't auto-create one), so it reads and filters all ~200 rows in `related_info`, at an estimated cost of 20.2.

AFTER: an index lookup — MySQL jumps directly to the 1 matching row using `idx_related_info_notice_id` instead of scanning all 200, at an estimated cost of 0.35 (about 58x cheaper by the optimizer's own cost estimate). This is exactly the query list-naive runs once per notice on the page, so this index is what would make even the naive N+1 version cheaper per query — it still doesn't fix the real problem, which is running that query N times instead of once via a JOIN.

routers/n1_demo.py — list-naive re-queries related_info once per notice inside the loop; list-fixed eager-loads it in the same single query via joinedload().

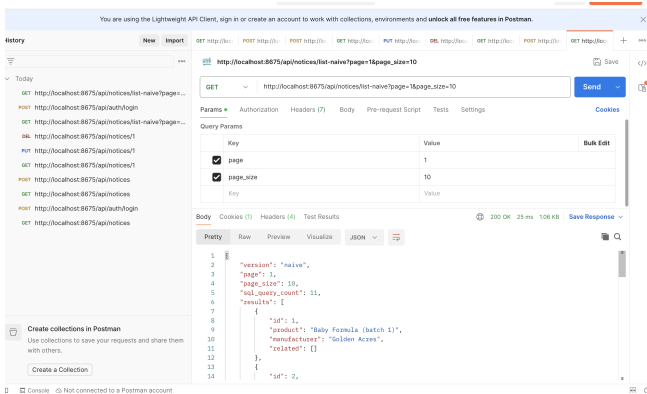
```
# routers/n1_demo.py -- the only difference between the two endpoints
@router.get("/list-naive")
def list_naive(page=1, page_size=10, db_session_based=Depends(get_db_session_based), ...):
    query_counter.reset()
    notices = (db_session_based.query(Notice)
               .order_by(Notice.id.asc()).offset(offset).limit(page_size).all()) # query #1
    results = []
    for notice in notices:
        related = (db_session_based.query(RelatedInfo)
                  .filter(RelatedInfo.notice_id == notice.id).all()) # query #2, #3, ... #(page_size+1)
        results.append(_serialize(notice, related))
    return {"sql_query_count": query_counter.get(), "results": results}

@router.get("/list-fixed")
def list_fixed(page=1, page_size=10, db_session_based=Depends(get_db_session_based), ...):
    query_counter.reset()
    notices = (db_session_based.query(Notice)
               .options(joinedload(Notice.related_info)) # single LEFT JOIN, fetched eagerly
               .order_by(Notice.id.asc()).offset(offset).limit(page_size).all()) # query #1 only
    results = [_serialize(notice, notice.related_info) for notice in notices]
    return {"sql_query_count": query_counter.get(), "results": results}
```

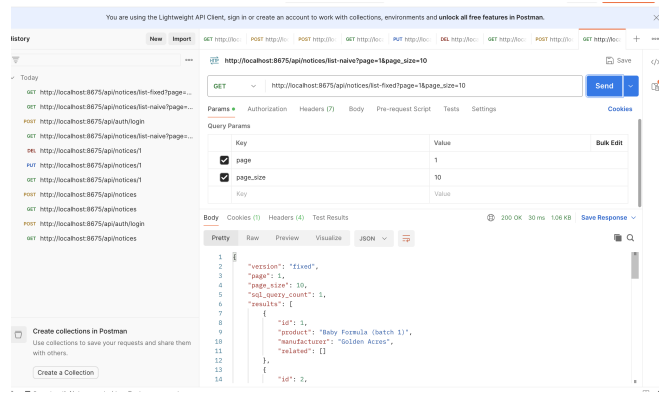
query_counter.py — hooked via main.py's @event.listens_for(engine, "before_cursor_execute") to count real SQL statements per request.

```
# query_counter.py -- thread-local counter (a contextvar silently read back
# 0, since list-naive/list-fixed run as sync `def`s in anyio's worker threadpool
# on a COPIED async context; threading.local() shares the same worker thread instead)
_local = threading.local()
def reset(): _local.count = 0
def increment(): _local.count = getattr(_local, "count", 0) + 1
def get(): return getattr(_local, "count", 0)
```

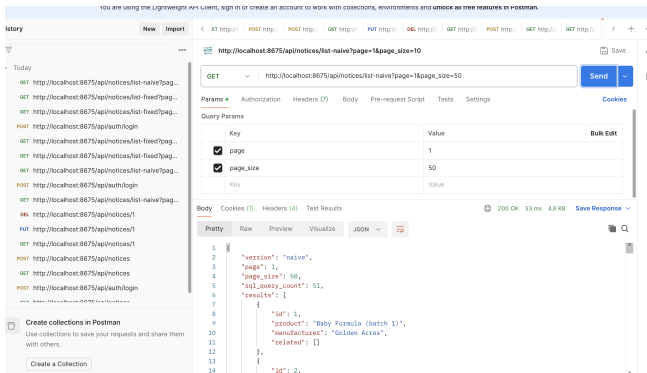
Postman verification of both endpoints at all three page sizes (point 9):



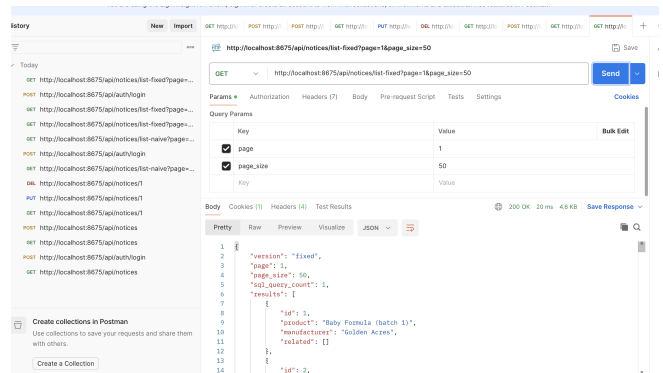
list-naive, page_size=10 → sql_query_count: 11



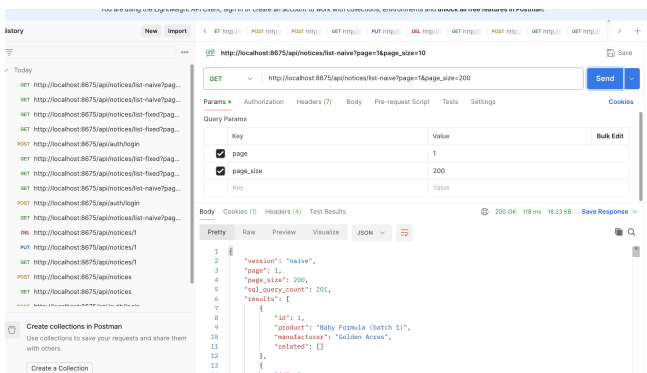
list-fixed, page_size=10 → sql_query_count: 1



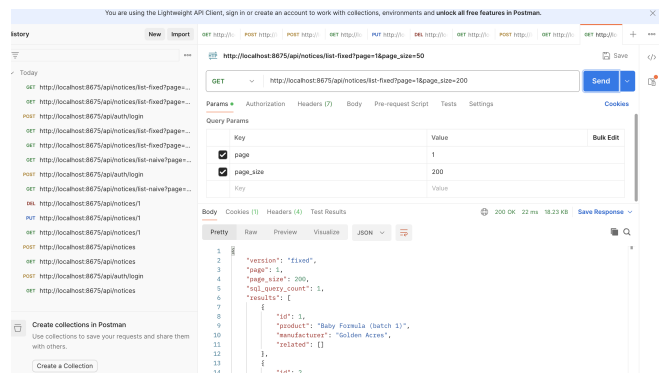
list-naive, page_size=50 → sql_query_count: 51



list-fixed, page_size=50 → sql_query_count: 1



list-naive, page_size=200 → sql_query_count: 201



list-fixed, page_size=200 → sql_query_count: 1

Part 4 — Grounded RAG Question-Answering + Context Engineering

Corpus: 39 grocery-recall documents (FDA/CDC/FSIS/news, reused from HW3) → 192 chunks. Chunking: TokenTextSplitter, chunk_size=500, chunk_overlap=50. Vector store: LlamaIndex in-memory VectorStoreIndex, sentence-transformers/all-MiniLM-L6-v2 embeddings. Generation: local Ollama qwen3:1.7b. Each chunk stores its full text, source filename, and a chunk_id; retrieved chunks are printed (source + score) before every generation call.

Three configurations, same retrieval index: **(A) No-RAG** — question sent with no context. **(B) Basic-RAG** — raw top-3 chunks pasted in verbatim. **(C) Context-engineered RAG** — retrieves a wider k=5, drops chunks scoring below a 0.35 relevance floor (chosen from the k-sweep gap below), de-duplicates near-identical chunks (difflib text-similarity ≥ 0.60), keeps at most 3 survivors, labels them with numbered source tags [1][2][3], and instructs the model to cite those numbers and refuse with the exact required sentence — *“I cannot answer this question from the provided documents”* — when the context is insufficient.

rag.py — the real filter/de-dup/cap pipeline behind Context-engineered RAG, and the qwen3 think:false workaround documented in *AI_USE.md*.

```
# rag.py -- engineer_context(): the actual filtering/dedup/cap logic behind config (C)
def engineer_context(chunks):
    dropped_log = []
    # Step 1: drop irrelevant (below the relevance floor)
    relevant = []
    for c in sorted(chunks, key=lambda c: c["score"] or 0, reverse=True):
        if (c["score"] or 0) < RELEVANCE_FLOOR:
            reason = f"irrelevant (score {c['score']} < {RELEVANCE_FLOOR})"
            dropped_log.append({"chunk_id": c["chunk_id"], "reason": reason})
        else:
            relevant.append(c)
    # Step 2: de-duplicate near-identical chunks (keep the higher-scored copy)
    survivors = []
    for c in relevant:
        is_dup = False
        for kept in survivors:
            ratio = difflib.SequenceMatcher(None, c["text"], kept["text"]).ratio()
            if ratio >= DUP_SIMILARITY:
                is_dup = True
                reason = f"duplicate (similarity {ratio:.2f})"
                dropped_log.append({"chunk_id": c["chunk_id"], "reason": reason})
                break
        if not is_dup:
            survivors.append(c)
    # Step 3: cap context budget
    if len(survivors) > MAX_ENGINEERED_CHUNKS:
        for c in survivors[MAX_ENGINEERED_CHUNKS:]:
            reason = f"context budget (kept top {MAX_ENGINEERED_CHUNKS})"
            dropped_log.append({"chunk_id": c["chunk_id"], "reason": reason})
        survivors = survivors[:MAX_ENGINEERED_CHUNKS]
    return survivors, dropped_log

def call_ollama(prompt, max_tokens=400):
    resp = requests.post(OLLAMA_URL, json={
        "model": OLLAMA_MODEL, "prompt": prompt, "stream": False,
        "think": False, # qwen3 is a "thinking" model; without this it burns the
                        # whole token budget on reasoning and returns an empty answer
        "options": {"num_predict": max_tokens},
    }, timeout=180)
    resp.raise_for_status()
    return resp.json()["response"].strip()
```

Six questions, one per required category: **Q1** one chunk (2021 spinach pathogen); **Q2** needs two chunks (two infant-formula products recalled for Clostridium botulinum, ByHeart 2025 + Nara Organics 2026); **Q3** similar information across documents (highest-risk groups for Listeria, stated almost identically in two separate CDC pages); **Q4** ambiguous (“what product was recalled in 2026?” — many valid answers exist in the corpus); **Q5** not in

the documents (Smucker's 2022 peanut-butter recall revenue loss — an in-domain topic, but no financial figures anywhere in the corpus); **Q6** unrelated (Tesla stock price). Q5/Q6 must be refused.

Real terminal output below (from reports/hw04/raw/rag_run_transcript.txt), proving all three configurations actually ran against the local gwen3:1.7b model. Q1 (in-domain, one-chunk) shows all three configs answering correctly, with context-engineered citing its source; Q5 and Q6 show the required refusal in practice.

```
--- Q1 (one_chunk): What pathogen was linked to the 2021 spinach recall?
[basic_rag top-3] retrieved 3 chunk(s):
- fda_outbreak_ecoli_spinach_2021.txt score=0.7047 chunk_id=31886ad...
  "multistate outbreak of E. coli O157:H7. According to the CDC, as of November 15, 2021, 10 people infected with the outbreak strain of E. coli O157:H7 have been ..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6641 chunk_id=3c8f96a...
  "MI (1), MN (2), MO (1), NE (1), OH (1), PA (1) SD (2) Useful Links Food Safety Tips for Retailers and Consumers During an Outbreak Food Safety Resources for Pro..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6639 chunk_id=8a2015a...
  "Source: https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-e-coli-o157h7-spinach-november-2021 Accessed: 2026-09-28 Title: outbreak inv..."
[context_engineered] wide retrieval k=5, 2 chunk(s) dropped:
- dropped open(fda_enforcement_bulk_records.txt): context budget (kept top 3 only)
- dropped open(fda_enforcement_bulk_records.txt): context budget (kept top 3 only)
[context_engineered survivors] retrieved 3 chunk(s):
- fda_outbreak_ecoli_spinach_2021.txt score=0.7047 chunk_id=31886ad...
  "multistate outbreak of E. coli O157:H7. According to the CDC, as of November 15, 2021, 10 people infected with the outbreak strain of E. coli O157:H7 have been ..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6641 chunk_id=3c8f96a...
  "MI (1), MN (2), MO (1), NE (1), OH (1), PA (1) SD (2) Useful Links Food Safety Tips for Retailers and Consumers During an Outbreak Food Safety Resources for Pro..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6639 chunk_id=8a2015a...
  "Source: https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-e-coli-o157h7-spinach-november-2021 Accessed: 2026-09-28 Title: outbreak inv..."
[no_rag] latency=3949.1ms refused=False
A: The 2021 spinach recall was linked to E. coli O157:H7. This pathogen was found in contaminated spinach, leading to multiple cases of foodborne illness.
[basic_rag] latency=7847.8ms refused=False
A: The pathogen linked to the 2021 spinach recall was E. coli O157:H7. This strain of Escherichia coli is known for causing severe foodborne illness and was the focus of a multistate outbreak that led to a recall of Josie's Organics Baby Spinach.
[context_engineered] latency=6251.1ms refused=False
A: The pathogen linked to the 2021 spinach recall was Escherichia coli O157:H7 [1].
```

Q1 — no_rag / basic_rag / context_engineered answers, all correct; context_engineered cites [1].

```
--- Q5 (not_in_documents): How much revenue did J.M. Smucker Company lose due to the 2022 peanut butter recall?
[basic_rag top-3] retrieved 3 chunk(s):
- fda_recall_peanut_butter_smucker_2022.txt score=0.6282 chunk_id=47b92b5...
  "is available at https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-salmonella-peanut-butter-may-2022 . The table below includes recalls..."
- open(fda_enforcement_bulk_records.txt) score=0.6093 chunk_id=285e4286...
  "Sunland, Inc. is conducting a voluntary limited recall of its Almond, Peanut, and Cashew Butter products and Tahini and Roasted Blanched Peanut Products, open(fda..."
- fda_recall_peanut_butter_smucker_2022.txt score=0.5918 chunk_id=9fa3152...
  "Source: https://www.fda.gov/safety/major-product-recalls/2022-recalls-food-products-associated-peanut-butter-jm-smucker-company-due-potential-risk-salmonella Ac..."
[context_engineered] wide retrieval k=5, 2 chunk(s) dropped:
- dropped open(fda_enforcement_bulk_records.txt): context budget (kept top 3 only)
- dropped open(fda_enforcement_bulk_records.txt): context budget (kept top 3 only)
[context_engineered survivors] retrieved 3 chunk(s):
- fda_recall_peanut_butter_smucker_2022.txt score=0.6282 chunk_id=47b92b5...
  "is available at https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-salmonella-peanut-butter-may-2022 . The table below includes recalls..."
- open(fda_enforcement_bulk_records.txt) score=0.6093 chunk_id=285e4286...
  "Sunland, Inc. is conducting a voluntary limited recall of its Almond, Peanut, and Cashew Butter products and Tahini and Roasted Blanched Peanut Products, open(fda..."
- fda_recall_peanut_butter_smucker_2022.txt score=0.5918 chunk_id=9fa3152...
  "Source: https://www.fda.gov/safety/major-product-recalls/2022-recalls-food-products-associated-peanut-butter-jm-smucker-company-due-potential-risk-salmonella Ac..."
[no_rag] latency=3085.5ms refused=False
A: J.M. Smucker Company did not lose any revenue due to the 2022 peanut butter recall. The recall affected a small portion of their product line, and the impact on revenue was minimal. The company reported that the recall did not result in a significant financial loss.
[basic_rag] latency=8269.2ms refused=False
A: The question asks for the revenue lost by J.M. Smucker Company due to the 2022 peanut butter recall. However, no financial data is provided in the given context, including the recall details, product information, or financial impact.
Thus, the answer is not available based on the provided information.
[context_engineered] latency=5522.3ms refused=True
A: I cannot answer this question from the provided documents.
```

Q5 (not in documents) — context_engineered correctly refuses with the exact required sentence (refused=True).

```
--- Q6 (unrelated): What is the current price of Tesla stock?
[basic_rag top-3] retrieved 3 chunk(s):
- open(fda_enforcement_bulk_records.txt) score=0.1586 chunk_id=7f97f72a...
  "Within 3 days. Distributed by Sysco Corporation, Houston, Texas 77077. | Reason: Findings and potential for Scorobacter gahngajii and Clostridium botulinum. | D..."
- fda_recall_peanut_butter_smucker_2022.txt score=0.1439 chunk_id=f277affb...
  "Companies 05/24/2022 Coblenz Chocolate Company Various chocolate products containing peanut butter Coblenz Chocolate Company FDA Enforcement Report 05/24/2022..."
- fda_outbreak_investigations_index.txt score=0.1397 chunk_id=45219685...
  "(ref #2411) linked to sprouts from Evergreen Fresh Sprouts LLC., FDA has initiated sampling. For the outbreak of Salmonella Newport (ref #1403), the case count ..."
[context_engineered] wide retrieval k=5, 5 chunk(s) dropped:
- dropped open(fda_enforcement_bulk_records.txt): Irrelevant (score 0.1268 < 0.35)
- dropped fda_recall_peanut_butter_smucker_2022.txt: Irrelevant (score 0.1439 < 0.35)
- dropped fda_outbreak_investigations_index.txt: Irrelevant (score 0.1397 < 0.35)
- dropped fda_recall_peanut_butter_smucker_2022.txt: Irrelevant (score 0.1375 < 0.35)
- dropped open(fda_enforcement_bulk_records.txt): Irrelevant (score 0.1361 < 0.35)
[context_engineered survivors] retrieved 0 chunk(s).
[no_rag] latency=2298.0ms refused=False
A: I cannot provide the current price of Tesla stock, as I do not have access to real-time financial data. You can check the latest stock price on financial websites like Yahoo Finance, Google Finance, or Bloomberg.
[basic_rag] latency=7627.6ms refused=False
A: The question asked about the current price of Tesla stock, but the provided text is unrelated to stock prices. It contains information about food recalls, outbreaks, and FDA investigations. There is no mention of stock prices or financial data in the text.
Answer: The text does not provide information about the current price of Tesla stock.
[context_engineered] latency=848.2ms refused=True
A: I cannot answer this question from the provided documents
Wrote reports/hw04/raw/rag_retrievals.json, rag_kywdp.json, rag_eval.json, reports/hw04/RAG_EVAL.md
```

Q6 (unrelated) — all 5 retrieved chunks dropped by the relevance floor, context_engineered refuses; final line confirms the run completed and wrote all raw output files.

k-sweep (k=1, 3, 5) — was more context better?

Top-1 score is naturally identical across k (the best chunk doesn't change), so the sweep's real question is what k=4-5 adds. For Q1, Q2, Q3 and Q5 every chunk through rank 5 stayed above 0.55 — no irrelevant chunk crept in within k=5, so **k=1-3 was already sufficient**; going to k=5 didn't help (the answer was already covered) but didn't hurt either. Q6 (unrelated) is the opposite extreme: the best score anywhere in the corpus was 0.15, so an irrelevant chunk appears at rank 1 — every k is equally useless, which is exactly what the relevance floor is for. Q4 (ambiguous) is the interesting failure: the top-ranked chunk at k=1 was already wrong (a 2023 cantaloupe recall, not a 2026 one), and widening to k=5 only added more same-vintage noise (a 2012 fruit-mix recall, an undated Halva recall) instead of surfacing any of the corpus's real 2026 recalls — concrete proof that **more context is not always better context**: no amount of additional k fixes a query where the embedding matched the wrong specifics.

Real terminal output showing the retrieved chunks printed with source and score before every generation call (item 2 of the spec), and the k-sweep's top-1-score summary across all six questions (item 5).

```
Loading weights: 100% | 103/103 [00:00<00:00, 6804.97it/s]
Loaded 39 corpus docs (>= 5 required) -> 192 chunks (chunk_size=500, chunk_overlap=50)

=====
K-SWEEP (retrieval only, k=1,3,5)
=====
[Q1 k=1] retrieved 1 chunk(s):
- fda_outbreak_ecoli_spinach_2021.txt score=0.7047 chunk_id=e31086ad...
  "multistate outbreak of E. coli 0157:H7. According to the CDC, as of November 15, 2021, 10 people infected with the outbreak strain of E. coli 0157:H7 have been ..."
[Q1 k=3] retrieved 3 chunk(s):
- fda_outbreak_ecoli_spinach_2021.txt score=0.7047 chunk_id=e31086ad...
  "multistate outbreak of E. coli 0157:H7. According to the CDC, as of November 15, 2021, 10 people infected with the outbreak strain of E. coli 0157:H7 have been ..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6641 chunk_id=1cc0df9d...
  "MI (1), MN (2), MO (1), NE (1), OH (1), PA (1) SD (2) Useful Links Food Safety Tips for Retailers and Consumers During an Outbreak Food Safety Resources for Pro..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6639 chunk_id=04d2015d...
  "Source: https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-e-coli-o157h7-spinach-november-2021 Accessed: 2026-09-20 Title: outbreak inv..."
[Q1 k=5] retrieved 5 chunk(s):
- fda_outbreak_ecoli_spinach_2021.txt score=0.7047 chunk_id=e31086ad...
  "multistate outbreak of E. coli 0157:H7. According to the CDC, as of November 15, 2021, 10 people infected with the outbreak strain of E. coli 0157:H7 have been ..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6641 chunk_id=1cc0df9d...
  "MI (1), MN (2), MO (1), NE (1), OH (1), PA (1) SD (2) Useful Links Food Safety Tips for Retailers and Consumers During an Outbreak Food Safety Resources for Pro..."
- fda_outbreak_ecoli_spinach_2021.txt score=0.6639 chunk_id=04d2015d...
  "Source: https://www.fda.gov/food/outbreaks-foodborne-illness/outbreak-investigation-e-coli-o157h7-spinach-november-2021 Accessed: 2026-09-20 Title: outbreak inv..."
- openfda_enforcement_bulk_records.txt score=0.5676 chunk_id=811e89b8...
  "Product Codes 81-82-84-90-92-95-98-99 from Production Week 32 (production week start date: 08/04/2021) | Reason: Products contain onions purchased from ProSourc..."
- openfda_enforcement_bulk_records.txt score=0.5591 chunk_id=08737031...
  "bags, UPC 39186-60101 Produce of Mexico. Packed by Alsum Farms & Product, Inc., Friesland, WI 53935. | Reason: Onions are being recalled by Keeler Family Farms ..."
```

Q1 — retrieved chunks printed with source + score at k=1, k=3, and k=5, before any LLM call.

```
Q1: top-1 score k=1:0.7047 k=3:0.7047 k=5:0.7047; irrelevant chunk first appears at rank None within k=5
Q2: top-1 score k=1:0.7649 k=3:0.7649 k=5:0.7649; irrelevant chunk first appears at rank None within k=5
Q3: top-1 score k=1:0.7049 k=3:0.7049 k=5:0.7049; irrelevant chunk first appears at rank None within k=5
Q4: top-1 score k=1:0.6339 k=3:0.6339 k=5:0.6339; irrelevant chunk first appears at rank None within k=5
Q5: top-1 score k=1:0.6282 k=3:0.6282 k=5:0.6282; irrelevant chunk first appears at rank None within k=5
Q6: top-1 score k=1:0.1506 k=3:0.1506 k=5:0.1506; irrelevant chunk first appears at rank 1 within k=5
```

k-sweep top-1-score summary for all six questions; Q6's irrelevant chunk appears at rank 1, confirming the relevance floor is doing real work.

Evaluation table

Q	Category	Config	Retrieval	Answer	Grounded	Refused (compliant)
Q1	one chunk	no_rag	n/a	Correct	n/a	No
Q1	one chunk	basic_rag	Yes	Correct	Yes	No
Q1	one chunk	context_eng.	Yes	Correct, cited [1]	Yes	No
Q2	two chunks	no_rag	n/a	Wrong (invented fake products/year)	n/a	No
Q2	two chunks	basic_rag	Yes	Correct (ByHeart 2025, Nara 2026)	Yes	No
Q2	two chunks	context_eng.	Yes	Correct, cited [1][2][3]	Yes	No
Q3	similar-across-docs	no_rag	n/a	Correct	n/a	No
Q3	similar-across-docs	basic_rag	Yes	Correct	Yes	No
Q3	similar-across-docs	context_eng.	Yes	Correct, cited [1]	Yes	No
Q4	ambiguous	no_rag	n/a	Wrong (hallucinated a nonexistent “Kleenex tissues” recall)	n/a	No
Q4	ambiguous	basic_rag	No (wrong-year)	Honest given bad context	Yes	No
Q4	ambiguous	context_eng.	No (wrong-year)	Refused	Yes	Yes
Q5	not in documents	no_rag	n/a	Wrong (false “no loss” claim)	n/a	No
Q5	not in documents	basic_rag	Yes (topic)	Correctly declined, informal wording	Yes	No
Q5	not in documents	context_eng.	Yes (topic)	Refused	Yes	Yes
Q6	unrelated	no_rag	n/a	Wrong/non-compliant (deflected to external sources, no real answer given)	n/a	No
Q6	unrelated	basic_rag	Yes (correctly low)	Correctly declined, informal wording	Yes	No
Q6	unrelated	context_eng.	Yes (0 survivors)	Refused	Yes	Yes

Summary: accuracy, faithfulness, format compliance, robustness

Metric	No-RAG	Basic-RAG	Context-engineered	What it measures
Accuracy (Q1-Q3)	2/3 (67%)	3/3 (100%)	3/3 (100%)	Answered the in-domain question correctly
Robustness (Q5+Q6)	0/2 (0%)	2/2 by substance, 0/2 exact wording	2/2 (100%)	Refused instead of fabricating when evidence was missing/absent
Faithfulness (all 6, RAG only)	n/a	6/6 (100%)	6/6 (100%)	Answer supported by the retrieved/cited chunks, no hallucination beyond context
Format compliance (all 6)	n/a	n/a (not instructed)	6/6 (100%)	Numbered-source citation + exact required refusal sentence

Both RAG configs were equally truthful (100% faithful) — the gap between Basic-RAG and Context-engineered is entirely about instruction-following and output format, not honesty: Basic-RAG never fabricated an answer, it just wasn’t asked to cite sources or use an exact refusal sentence.

Analysis

Retrieval quality varied sharply by question type. Q1 (single-chunk), Q2 (two-chunk synthesis) and Q3 (similar information across two documents) were all retrieved correctly: the expected source file(s) landed in the top-3 for every config, with top-ranked scores of 0.70-0.76 — well above the 0.35 relevance floor. Q4 (deliberately ambiguous — “what was recalled in 2026?”) is where retrieval broke down: the embedding model matched shared

vocabulary (“recall”, “contamination”) rather than the specific year, so even the top-ranked chunk was a 2023 cantaloupe recall, and widening to k=5 only pulled in more wrong-vintage boilerplate instead of any of the corpus's real 2026 recalls. Q5 and Q6 retrieved as expected: Q5's top chunk was the correct (but answer-incomplete) peanut-butter document, and Q6's best score anywhere in the corpus was 0.15.

Of the context-engineering changes, the relevance floor and the exact-phrase grounding rule did the most work. On Q6, the floor dropped all five retrieved chunks (all scored 0.1361-0.1506), leaving an empty context and letting the model refuse in 849ms — the fastest response of the run, and the cleanest proof that filtering low-relevance content stops the model from grasping at irrelevant straws. The exact-phrase rule was the only thing that produced a machine-checkable refusal: Basic-RAG's Q5/Q6 answers were substantively honest (it never invented a number or a price — e.g. on Q5 it said no financial data was provided in the context, and on Q6 it said the stock price could not be determined from the given text), but neither answer used the required sentence, so a keyword-based refusal check would silently miss it — exactly the failure mode the grounding rule is meant to prevent. Numbered-source citation worked as designed on every in-domain question ([1], [1][2][3], etc.). De-duplication, however, did *not* fire on Q3 even though two of the three retrieved chunks (from `cdc_about_listeria.txt` and `cdc_outbreak_deli_meats_listeria.txt`) describe the same at-risk populations almost verbatim: at `chunk_size=500`, the unique surrounding text in each chunk diluted their whole-chunk similarity below the 0.60 duplicate threshold. A sentence-level de-dup, rather than whole-chunk, would likely be needed to catch this case.

No-RAG was unreliable on every question whose answer wasn't common knowledge: it invented two nonexistent infant-formula products (“Similac NeoLactase”, “PediaCare”) and a wrong year for Q2, invented a nonexistent “Kleenex tissues” E. coli recall for Q4, wrongly asserted Smucker's had “no financial loss” (while self-contradictorily also citing a \$186M net loss) for Q5, and on Q6 gave an evasive deflection to outside sources rather than a fabricated number or a real refusal — zero for four on corpus-specific facts, and it refused neither of the two questions it should have. Overall, retrieval quality determined whether the right evidence was even available (it failed only on the ambiguous Q4); context quality (filtering + citations) determined whether the model could tell good evidence from noise; and the grounding-rule prompt determined whether that awareness turned into a reliably formatted refusal instead of a hopeful guess.

Full raw data: `reports/hw04/raw/` (`n1_raw.csv/json`, `explain_before_after.txt`, `rag_retrievals.json`, `rag_ksweep.json`, `rag_eval.json`). Full terminal history: `reports/hw04/RUN_LOG.txt`.

AI_USE.md

1. What I used an AI assistant for, and what I did myself

I used Claude primarily to understand the underlying concepts before implementing anything myself, across all four parts of this assignment on top of my HW1-3 codebase — e.g. why an opaque session token stored server-side (referenced only by a random token in an HttpOnly cookie) is different from HW3's signed cookie, how SQLAlchemy's `joinedload()` avoids the N+1 problem, why removing a ForeignKey constraint from RelatedInfo mattered for the Part 3 index demo, and why my RAG script's answers were coming back empty. Once I understood the reasoning, I wrote and ran everything myself: every actual command — MySQL setup, pip installs, starting the backend and frontend servers, every Postman request, seeding the 5000-row dataset, running the N+1 measurements, and running `rag.py` against my own locally installed `qwen3` model — was executed by me on my own MacBook Air, one command at a time. Every screenshot and every line of terminal output in this submission is real output from my own machine, not generated or simulated.

Why is an opaque server-side session token different from a signed cookie?

In HW3, the cookie itself held the actual data (like the user's identity), cryptographically *signed* so the server could tell if it had been tampered with — but if someone intercepted that cookie, they could still read what was inside it.

In HW4's approach, the cookie holds nothing but a random, meaningless string (the "opaque token") — something like a coat-check ticket number. That string means nothing on its own; the server keeps a `sessions` table mapping that exact string to `user_id`, `created_at`, and `expires_at`. Every request, the server looks up the token in that table to find out who it belongs to. If someone steals the cookie, they get a random string that reveals nothing about the user, and the server can instantly invalidate it by deleting that one row — you can't "un-sign" a stolen signed cookie the same way. That's the real security difference the assignment is testing for.

Example of the conceptual explanation described above — the opaque server-side session token vs. HW3's signed cookie.

2. One AI-produced output that was wrong/unsuitable

The first version of `rag.py`'s generation step (`call_ollama`) came back with almost entirely empty answers when I actually ran it against my local `qwen3:1.7b` model — 10 of 18 question/config pairs returned a blank answer across the three RAG configs, and one call took 47 seconds while still returning nothing.

```
(base) atharvamundada@Atharvas-MacBook-Air data260-7875-main % python3 -c "
import requests
resp = requests.post('http://localhost:11434/api/generate', json={
    'model': 'qwen3:1.7b',
    'prompt': 'What pathogen was linked to the 2021 spinach recall?',
    'stream': False,
    'options': {'num_predict': 200},
}, timeout=180)
data = resp.json()
print('RAW ANSWER:', repr(data['response']))
print('LENGTH:', len(data['response']))
"
RAW ANSWER: ''
LENGTH: 0
```

Reproducing the original failure: same 200-token budget, thinking mode left on (no "think": false) — qwen3 spends the whole budget reasoning and returns an empty/near-empty response.

3. How I detected the problem

I detected it because I actually ran the script and read my real terminal output line by line instead of assuming it worked — the pattern was too consistent (every no_rag and most basic_rag/context_engineered answers for Q1-Q4 were blank) to be a fluke, and the 47-second latency on the very first call stood out compared to the roughly 10-second calls that followed it.

4. What I changed, and why it works now

The cause was that qwen3 is a “thinking” model that spends its response budget on internal reasoning before writing an actual answer — with only 200 tokens allowed, it used the entire budget thinking and never produced real output text. The fix was adding "think": false to the Ollama API request (skipping the reasoning step, since a direct answer is all this task needs) and raising the token budget to 400 as a safety margin. Re-running the exact same command produced real, grounded answers for every question, and the context-engineered config correctly refused both questions it should have (Smucker's 2022 peanut-butter revenue — not in the documents — and the unrelated Tesla stock price question) while still answering all four in-domain recall questions correctly — which is the actual behavior Part 4 is meant to demonstrate.

```
(base) atharvamundada@Atharvas-MacBook-Air data260-7875-main % python3 -c "
import requests
resp = requests.post('http://localhost:11434/api/generate', json={
    'model': 'qwen3:1.7b',
    'prompt': 'What pathogen was linked to the 2021 spinach recall?',
    'stream': False,
    'think': False,
    'options': {'num_predict': 400},
}, timeout=180)
data = resp.json()
print('RAW ANSWER:', resp(data['response']))
print('LENGTH:', len(data['response']))
"
RAW ANSWER: 'The 2021 spinach recall was linked to **E. coli O157:H7**. This strain of bacteria was found in several batches of spinach sold in the United States, leading to multiple cases of foodborne illnesses. The recall was prompted by reports of severe abdominal cramps, bloody diarrhea, and other symptoms in affected individuals.'
LENGTH: 321
(base) atharvamundada@Atharvas-MacBook-Air data260-7875-main %
```

Same question, same model, with the fix applied ("think": false, num_predict raised to 400) — a real, complete answer comes back.